

LAPORAN TUGAS 1

SISTEM OPERASI



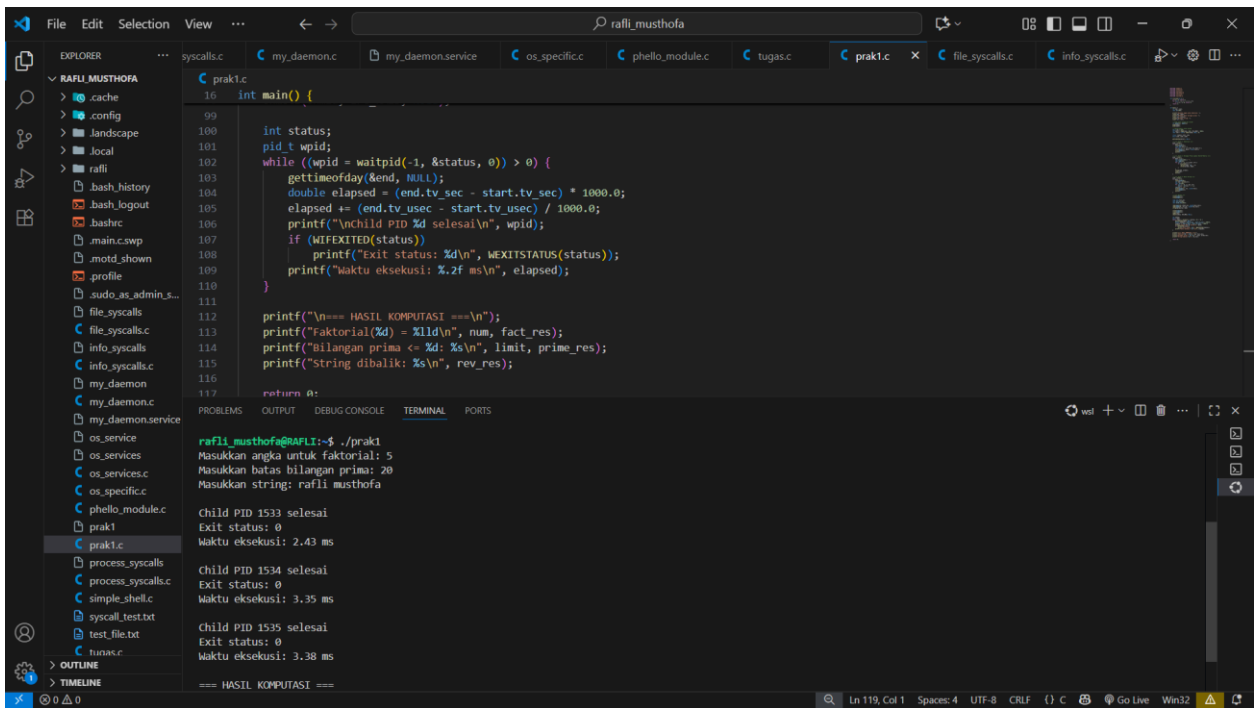
Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Rafli Musthofa (2410651019)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

Tugas 1

1. Implementasi



```
16 int main() {
99
100     int status;
101     pid_t wpid;
102     while ((wpid = waitpid(-1, &status, 0)) > 0) {
103         gettimeofday(&end, NULL);
104         double elapsed = (end.tv_sec - start.tv_sec) * 1000.0;
105         elapsed += (end.tv_usec - start.tv_usec) / 1000.0;
106         printf("\nchild PID %d selesai\n", wpid);
107         if (WIFEXITED(status))
108             printf("Exit status: %d\n", WEXITSTATUS(status));
109         printf("waktu eksekusi: %.2f ms\n", elapsed);
110     }
111
112     printf("\n=== HASIL KOMPUTASI ===\n");
113     printf("Faktorial(%d) = %lld\n", num, fact_res);
114     printf("Bilangan prima <= %d: %s\n", limit, prime_res);
115     printf("String dibalik: %s\n", rev_res);
116
117     return 0;
118 }
```

Terminal Output:

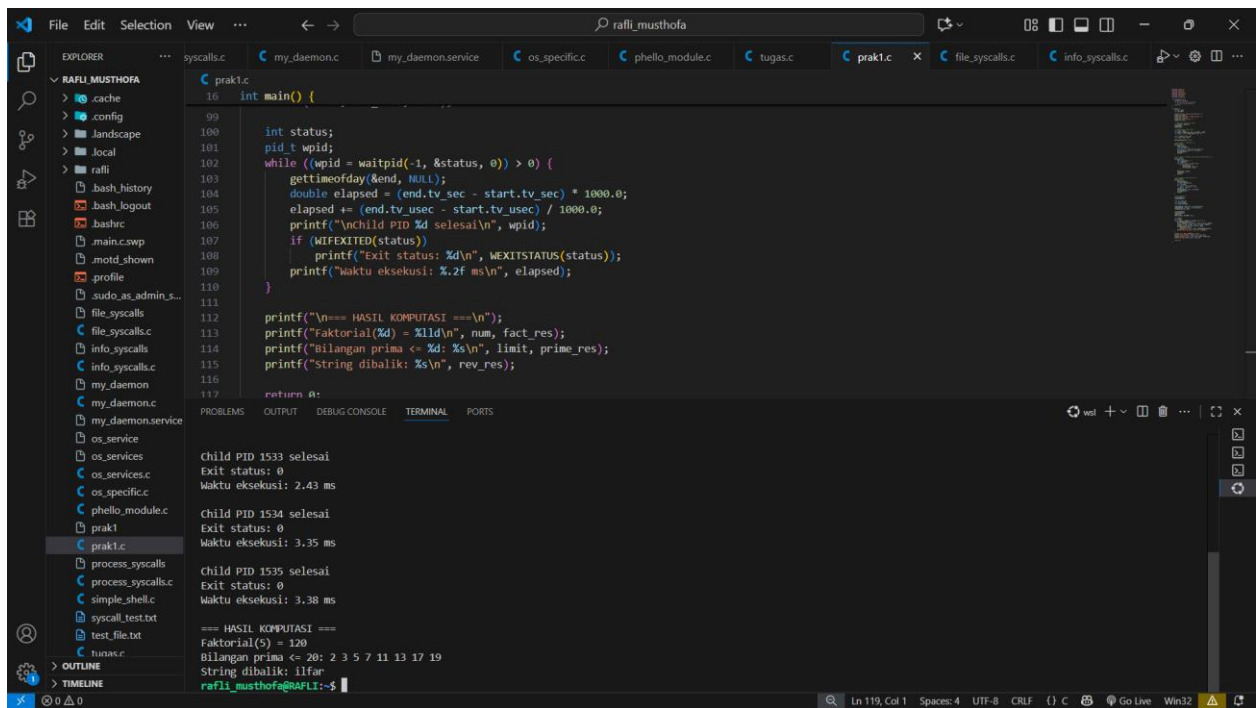
```
raflimusthofa@RAFLI:~$ ./prakt1
Masukkan angka untuk faktorial: 5
Masukkan batas bilangan prima: 20
Masukkan string: raflimusthofa

Child PID 1533 selesai
Exit status: 0
Waktu eksekusi: 2.43 ms

Child PID 1534 selesai
Exit status: 0
Waktu eksekusi: 3.35 ms

Child PID 1535 selesai
Exit status: 0
Waktu eksekusi: 3.38 ms

=== HASIL KOMPUTASI ===
```



```
Child PID 1533 selesai
Exit status: 0
Waktu eksekusi: 2.43 ms

Child PID 1534 selesai
Exit status: 0
Waktu eksekusi: 3.35 ms

Child PID 1535 selesai
Exit status: 0
Waktu eksekusi: 3.38 ms

=== HASIL KOMPUTASI ===
Faktorial(5) = 120
Bilangan prima <= 20: 2 3 5 7 11 13 17 19
String dibalik: 1flar
raflimusthofa@RAFLI:~$
```

2. Dokumentasi

/*

PRAKTIKUM SISTEM OPERASI – Implementasi Fork(), Pipe, dan Shared Memory
Dokumentasi tambahan: Struktur Virtual Filesystem /proc di Linux

/proc adalah *virtual filesystem* (bukan filesystem nyata di disk),
yang menyediakan informasi tentang kernel dan proses yang sedang berjalan.

/proc tidak menyimpan data secara permanen, melainkan menampilkan data
langsung dari kernel setiap kali file dibaca.

Contoh isi penting dari /proc:

/proc/cpuinfo → informasi CPU

/proc/meminfo → informasi memori (seperti perintah 'free')

/proc/stat → statistik CPU

/proc/<PID>/ → folder berisi info untuk setiap proses

/proc/uptime → waktu aktif sistem

/proc/loadavg → beban sistem

/proc/net/dev → statistik jaringan (mirip 'ifconfig')

/proc/filesystems → daftar filesystem yang dikenali kernel

Struktur umum:

```
/proc
├── 1/          → Proses init (PID 1)
├── 2/          → Proses kthreadd
├── 3/ ...
├── cpuinfo
├── meminfo
├── net/
│   └── dev
├── loadavg
└── uptime
```

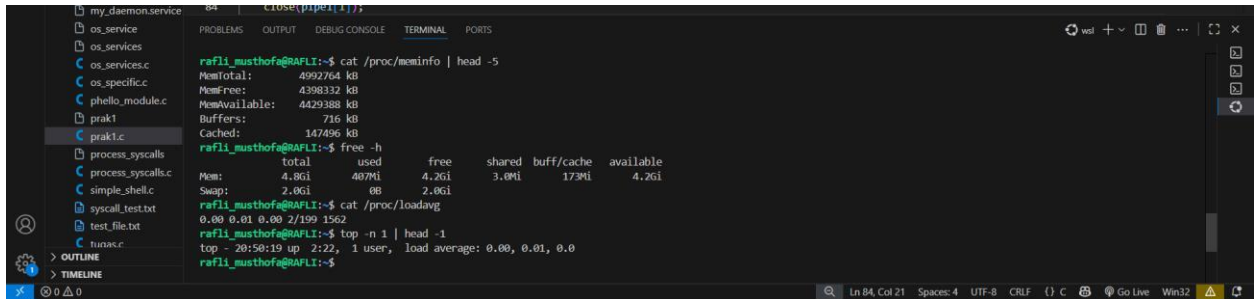
Error Handling:

Jika file /proc tidak dapat dibuka (misalnya permission denied),
program harus menampilkan pesan error menggunakan perror().
Pastikan file descriptor ditutup setelah digunakan.

*/

3. Pengujian

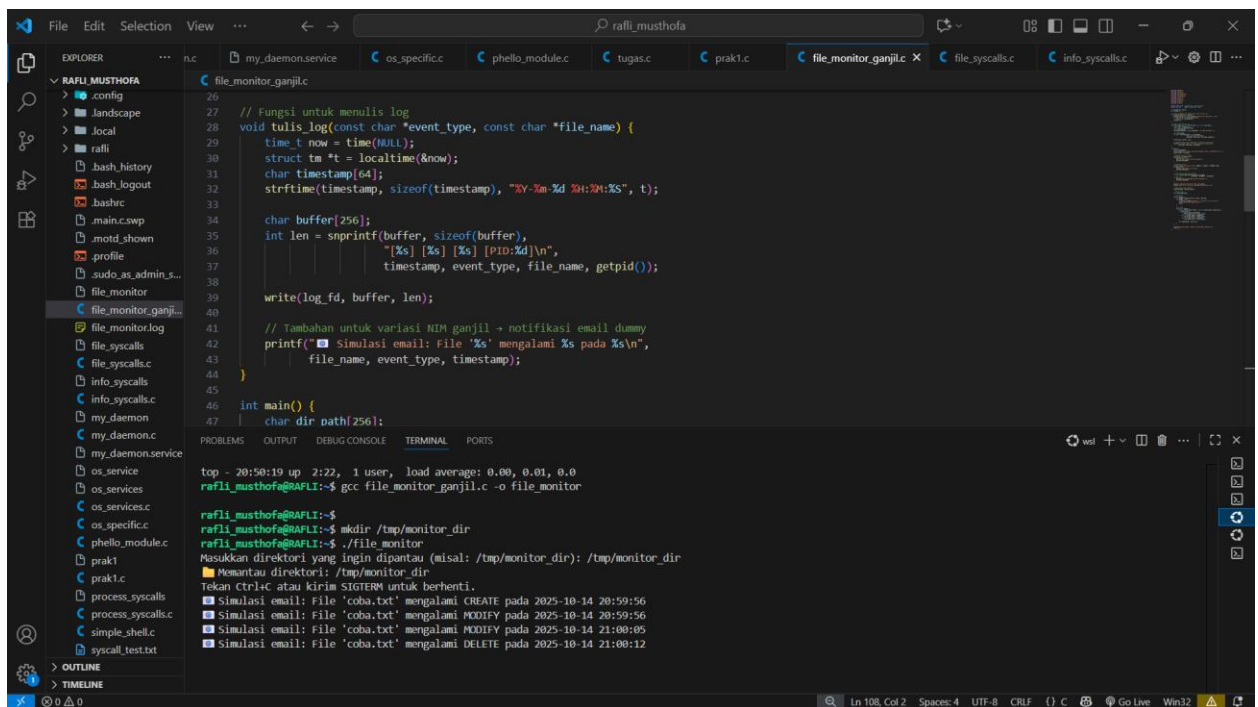
Contoh Pengujian:



```
rafli_musthofa@RAFLI:~$ cat /proc/meminfo | head -5
MemTotal: 4992764 kB
MemFree: 4398332 kB
MemAvailable: 4429388 kB
Buffers: 716 kB
Cached: 147496 kB
rafli_musthofa@RAFLI:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:            4.8Gi      407Mi       4.2Gi       3.0Mi       173Mi       4.2Gi
Swap:           2.0Gi          0B         2.0Gi
rafli_musthofa@RAFLI:~$ cat /proc/loadavg
0.00 0.01 0.00 2/199 1562
rafli_musthofa@RAFLI:~$ top -n 1 | head -1
top - 20:50:19 up 2:22, 1 user, load average: 0.00, 0.01, 0.0
rafli_musthofa@RAFLI:~$
```

Tugas 2

1. Implementasi



```
File Edit Selection View ... rafli_musthofa
EXPLORER ... n.c my_daemon.service os_specific.c phello_module.c tugas.c prak1.c file_monitor_ganjil.c file_syscalls.c info_syscalls.c
RAFLI MUSTHOFA
  .contig
  .landscape
  .local
  .rafl
  .bash_history
  .bash_logout
  .bashrc
  .main.c.swp
  .motd.shown
  .profile
  .sudo_as_admin_s...
  file_monitor
  file_monitor_ganjil.c
  file_monitor.log
  file_syscalls
  file_syscalls.c
  info_syscalls
  info_syscalls.c
  my_daemon
  my_daemon.c
  my_daemon.service
  os_service
  os_services
  os_services.c
  os_specific.c
  phello_module.c
  prak1
  prak1.c
  process_syscalls
  process_syscalls.c
  simple_shell.c
  syscall_test.txt
  syscall_test.txt

// Fungsi untuk menulis log
void tulis_log(const char *event_type, const char *file_name) {
    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char timestamp[64];
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", t);

    char buffer[256];
    int len = snprintf(buffer, sizeof(buffer),
        "[%s] [%s] [%s] [PID:%d]\n",
        timestamp, event_type, file_name, getpid());

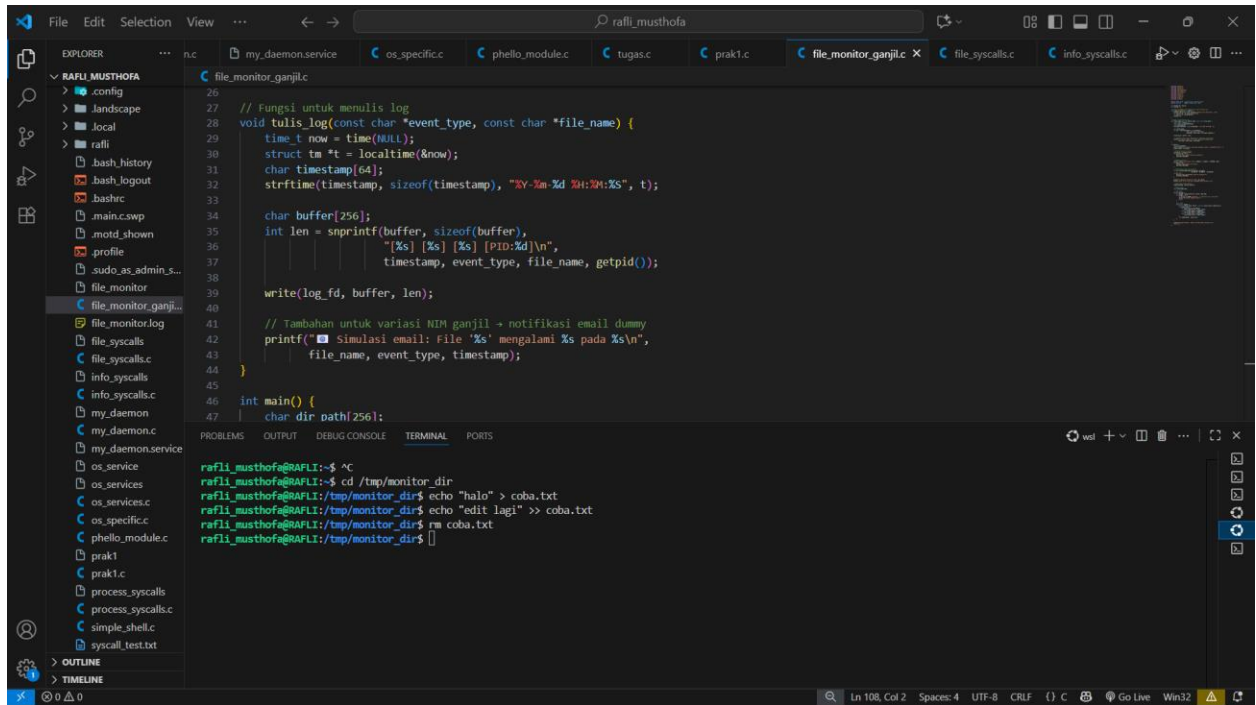
    write(log_fd, buffer, len);

    // Tambahkan untuk variasi NIM ganjil + notifikasi email dummy
    printf("Simulasi email: File '%s' mengalami %s pada %s\n",
        file_name, event_type, timestamp);
}

int main() {
    char dir_path[256];

top - 20:50:19 up 2:22, 1 user, load average: 0.00, 0.01, 0.0
rafli_musthofa@RAFLI:~$ gcc file_monitor_ganjil.c -o file_monitor

rafli_musthofa@RAFLI:~$
rafli_musthofa@RAFLI:~$ mkdir /tmp/monitor_dir
rafli_musthofa@RAFLI:~$ ./file_monitor
Pilihkan direktori yang ingin dipantau (misal: /tmp/monitor_dir): /tmp/monitor_dir
Masukkan direktori /tmp/monitor_dir
Tekan Ctrl+C atau Kirim SIGTERM untuk berhenti.
Simulasi email: File 'coba.txt' mengalami CREATE pada 2025-10-14 20:59:56
Simulasi email: File 'coba.txt' mengalami MODIFY pada 2025-10-14 20:59:56
Simulasi email: File 'coba.txt' mengalami MODIFY pada 2025-10-14 21:00:05
Simulasi email: File 'coba.txt' mengalami DELETE pada 2025-10-14 21:00:12
```



2. Documentasi

/*

=====

Struktur Data: struct inotify_event

=====

Struktur ini digunakan oleh kernel untuk mengirimkan informasi event ke user space melalui system call read().

Definisi (di <sys/inotify.h>):

```

struct inotify_event {
    int    wd;    // Watch descriptor yang dikembalikan dari inotify_add_watch()
    uint32_t mask; // Bitmask jenis event (misal IN_CREATE, IN_DELETE,
IN_MODIFY)
    uint32_t cookie; // Digunakan untuk event yang berpasangan (misal rename)
    uint32_t len;    // Panjang nama file (0 jika tidak ada nama)
    char    name[]; // Nama file (string null-terminated)
};

```

=====

Cara Membaca Buffer Event:

1. Kernel menulis satu atau lebih event ke buffer.

2. Masing-masing event memiliki ukuran: `EVENT_SIZE + event->len`
3. Program membaca semua event dengan while-loop, memindahkan offset sebesar ukuran event setiap kali.
4. Contoh implementasi:

```
int i = 0;
while (i < length) {
    struct inotify_event *event = (struct inotify_event*)&buffer[i];
    if (event->len) {
        // Proses event
    }
    i += EVENT_SIZE + event->len; // Lanjut ke event berikutnya
}
```

Error Handling:

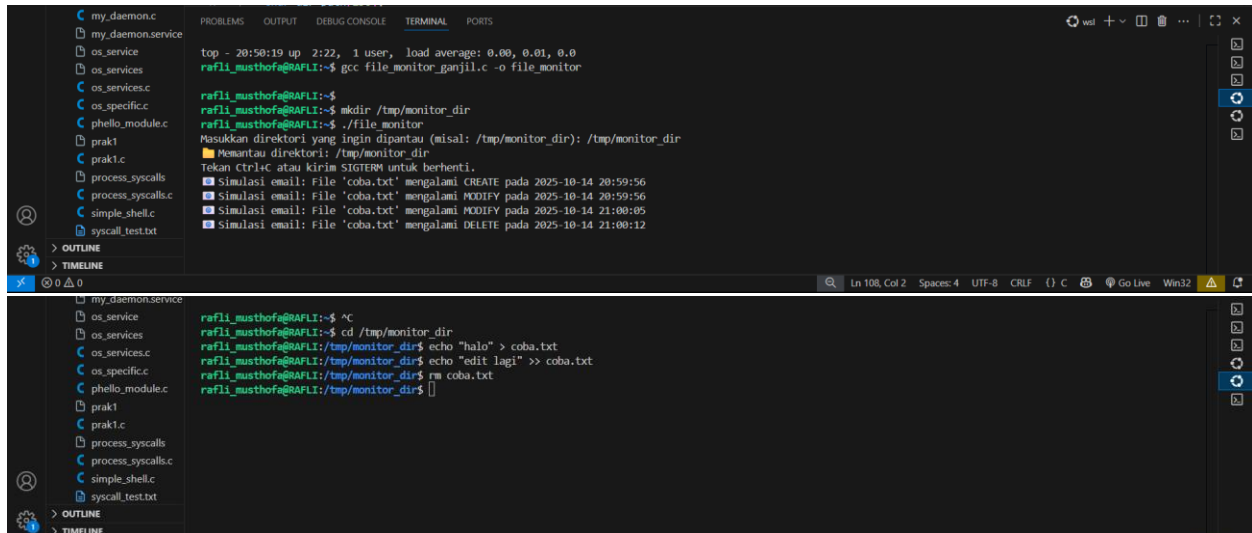
- Jika `read() < 0` dan `errno == EINTR` → berarti interupsi sinyal, abaikan dan lanjut.
- Jika muncul `ENOSPC` → jumlah watch inotify melebihi batas kernel.

Solusi: tingkatkan limit dengan:

```
sudo sysctl fs.inotify.max_user_watches=524288
```

```
*/
```

3. Pengujian



```
my_daemon.c
my_daemon.service
os_service
os_services
os_services.c
os_specific.c
phello_module.c
prakt1
prakt1.c
process_syscalls
process_syscalls.c
simple_shell.c
syscall_test.txt

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

top - 20:58:19 up 2:22, 1 user, load average: 0.00, 0.01, 0.0
rafl1_musthofa@RAFLI:~$ gcc file_monitor_ganjil.c -o file_monitor

rafl1_musthofa@RAFLI:~$
rafl1_musthofa@RAFLI:~$ mkdir /tmp/monitor_dir
rafl1_musthofa@RAFLI:~$ ./file_monitor
Masukkan direktori yang ingin dipantau (misal: /tmp/monitor_dir): /tmp/monitor_dir
Memantau direktori: /tmp/monitor_dir
Tekan Ctrl+C atau kirim SIGTERM untuk berhenti.
[+] Simulasi email: File 'coba.txt' mengalami CREATE pada 2025-10-14 20:59:56
[+] Simulasi email: File 'coba.txt' mengalami MODIFY pada 2025-10-14 20:59:56
[+] Simulasi email: File 'coba.txt' mengalami MODIFY pada 2025-10-14 21:00:05
[+] Simulasi email: File 'coba.txt' mengalami DELETE pada 2025-10-14 21:00:12

rafl1_musthofa@RAFLI:~$ ^C
rafl1_musthofa@RAFLI:~$ cd /tmp/monitor_dir
rafl1_musthofa@RAFLI:/tmp/monitor_dir$ echo "halo" > coba.txt
rafl1_musthofa@RAFLI:/tmp/monitor_dir$ echo "edit lagi" >> coba.txt
rafl1_musthofa@RAFLI:/tmp/monitor_dir$ rm coba.txt
rafl1_musthofa@RAFLI:/tmp/monitor_dir$
```

Tugas 3

1. Implementasi

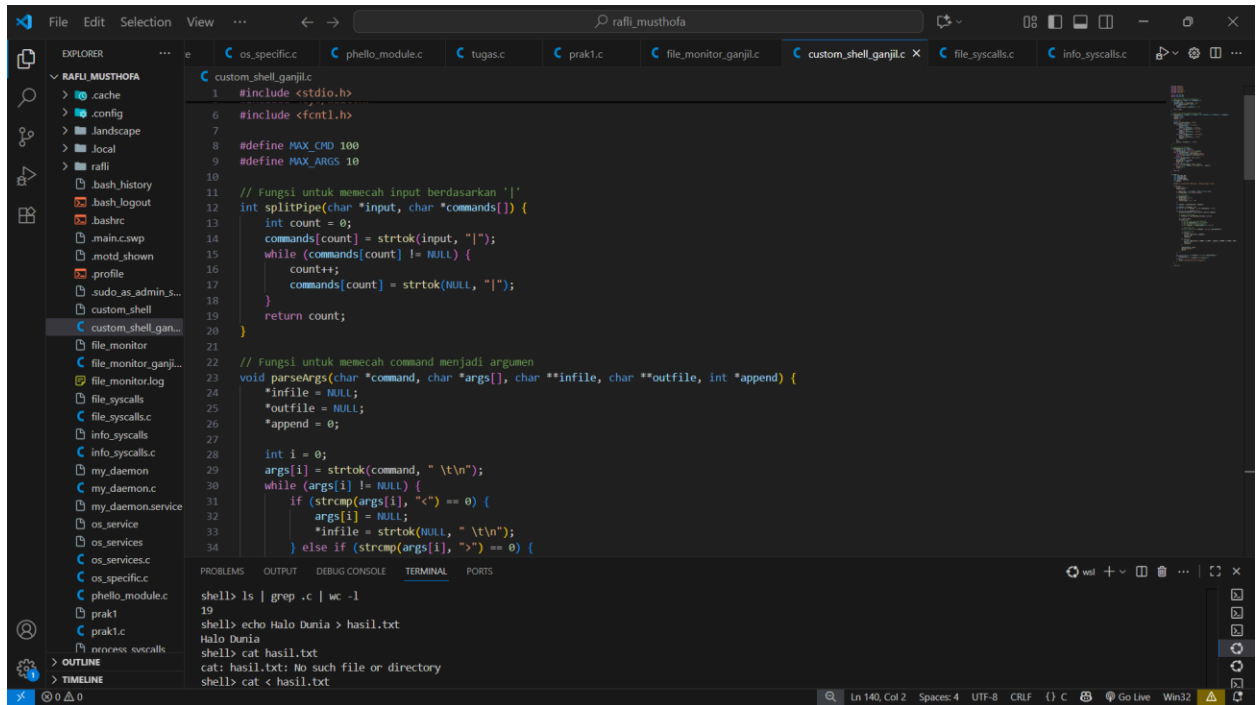
The image displays two screenshots of a Visual Studio Code editor window, showing the implementation of a custom shell named 'custom_shell_ganjil.c'.

Top Screenshot:

- Explorer:** Shows the project structure for 'RAFII_MUSTHOFA', including files like .cache, .config, .bash_history, .bash_logout, .bashrc, .main.c.swp, .motd_shown, .profile, .sudo_as_admin_s..., custom_shell, custom_shell_ganjil.c, file_monitor, file_monitor.ganji..., file_monitor.log, file_syscalls, file_syscalls.c, info_syscalls, info_syscalls.c, my_daemon, my_daemon.c, my_daemon.service, os_service, os_services, os_services.c, os_specific.c, phello_module.c, prak1, prak1.c, and nprocess_syscalls.
- Editor:** Displays the source code for 'custom_shell_ganjil.c'. The code defines a 'main()' function that enters a loop to handle multiple pipes. It uses 'fork()' to create child processes, 'wait(NULL)' to wait for them, and 'close(pipefd[j])' to manage the pipe descriptors. A 'printf()' statement indicates when a background process is running.
- Terminal:** Shows the execution of the shell. The prompt is 'shell>'. The user enters 'ls | grep .c | wc -l', and the output is '19'. The user then enters 'echo Halo Dunia > hasil.txt', and the output is 'Halo Dunia'. The user enters 'cat hasil.txt', and the output is 'cat: hasil.txt: No such file or directory'. The user enters 'cat < hasil.txt', and the output is 'halo saya rafii musthofa'.

Bottom Screenshot:

- Explorer:** Same project structure as the top screenshot.
- Editor:** Same source code for 'custom_shell_ganjil.c'.
- Terminal:** Shows the execution of the shell. The prompt is 'rafii_musthofa@RAFII:~\$./custom shell'. The user enters 'pad', and the output is 'pad'. The user enters 'ls | grep .c', and the output is 'execvp: No such file or directory'. The user enters 'ls | grep .c | wc -l', and the output is 'execvp: No such file or directory'. The user enters 'pad', and the output is '0'. The user enters 'cat hasil.txt', and the output is 'Halo Dunia'. The user enters 'cat < hasil.txt', and the output is 'halo saya rafii musthofa'.



2. Document

1. Penjelasan Parsing Logic dalam Kode

Komentar utama dalam kode (yang bisa kamu tambahkan langsung di custom_shell_ganjil.c):

// Parsing logic:

// 1. Baca input dengan fgets()

// 2. Pecah string input berdasarkan '|' untuk deteksi multiple pipe

// 3. Setiap bagian pipe dipecah lagi dengan strtok() menjadi argv[] untuk execvp()

// 4. Jika ada '&', tandai sebagai background process dan hilangkan dari argv

// 5. Jika ada '>' atau '<', lakukan redirection dengan open() dan dup2()

// 6. Jalankan perintah via fork() + execvp()

2. Penjelasan Bagaimana Pipe Mengubah File Descriptor

// Pipe mengubah file descriptor standar:

// - pipefd[0] = ujung untuk membaca (stdin)

// - pipefd[1] = ujung untuk menulis (stdout)

// dup2(pipefd[1], STDOUT_FILENO) -> output perintah pertama masuk ke pipe

// dup2(pipefd[0], STDIN_FILENO) -> input perintah kedua berasal dari pipe

// Contoh: ls | grep txt

// [ls stdout -> pipe write] ==> [pipe read -> grep stdin]

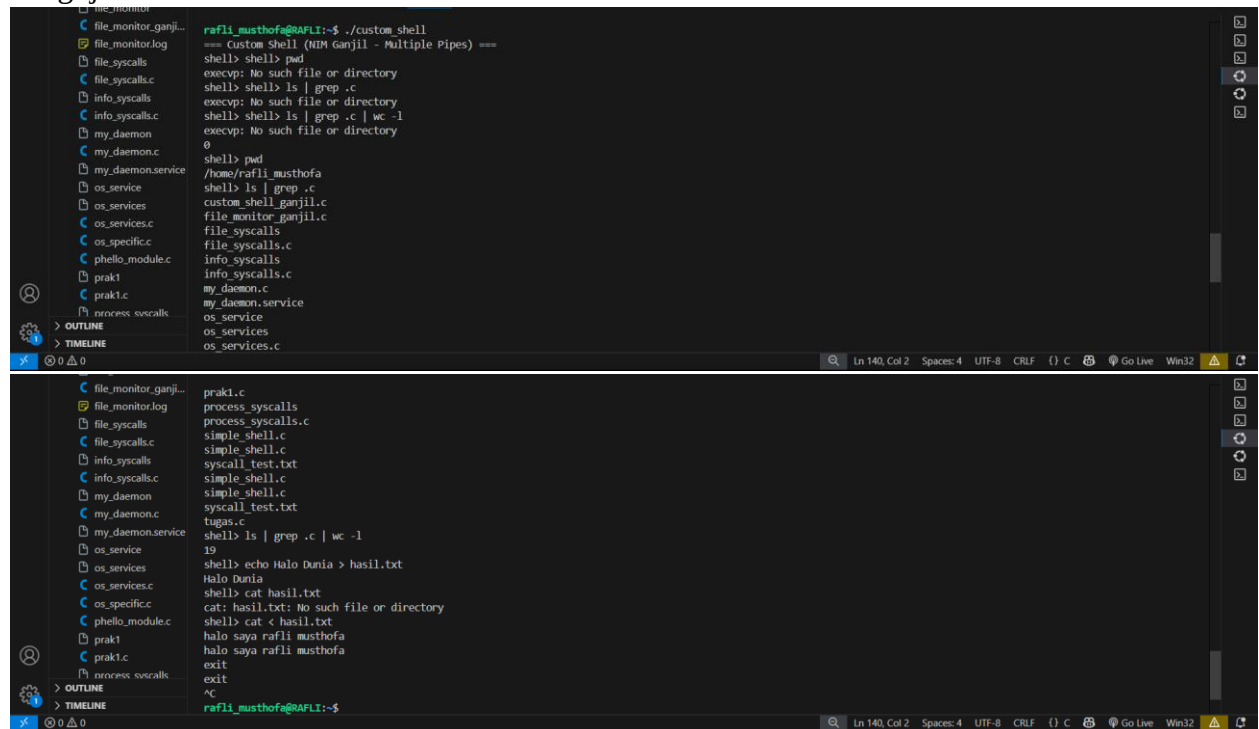
Intinya:

- dup2() mengganti standar input/output process child.
- Tanpa pipe: stdout = terminal.
- Dengan pipe: stdout dialihkan ke pipe, lalu dibaca oleh process berikutnya.

3. Kesimpulan Dokumentasi

- pipe() dan dup2() memungkinkan komunikasi antar-proses lewat FD.
- fork() + execvp() menjadi dasar pembuatan shell sederhana.
- Redirection (>, <, >>) cukup menggunakan kombinasi open() + dup2().
- Background process tetap perlu SIGCHLD handling agar tidak jadi zombie.
- Shell custom ini menggambarkan konsep I/O redirection dan process hierarchy di Linux.

3. Pengujian

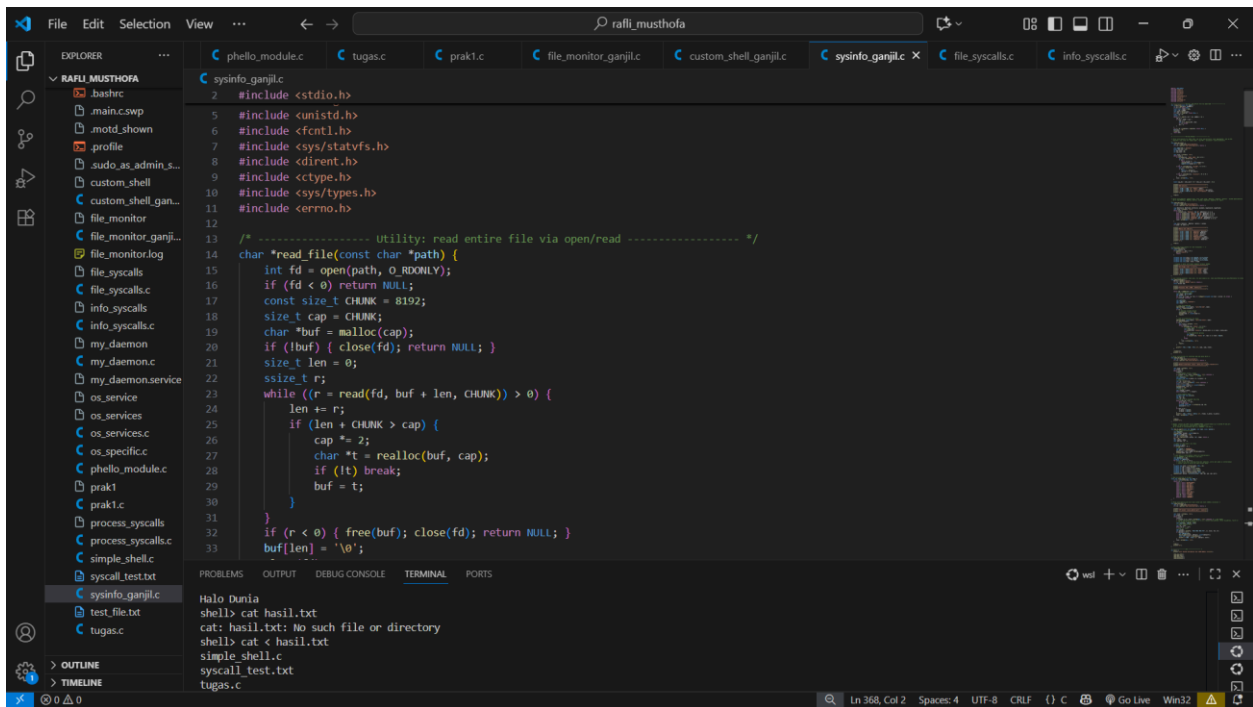


```
rafl_i_musthofa@RAFLI:~$ ./custom_shell
=== Custom Shell (MIM Ganjil - Multiple Pipes) ===
shell> shell> pwd
execvp: No such file or directory
shell> shell> ls | grep .c
execvp: No such file or directory
shell> shell> ls | grep .c | wc -l
execvp: No such file or directory
0
shell> pwd
/home/rafl_i_musthofa
shell> ls | grep .c
custom_shell_ganjil.c
file_monitor_ganjil.c
file_syscalls
file_syscalls.c
info_syscalls
info_syscalls.c
my_daemon
my_daemon.c
my_daemon.service
os_service
os_services
os_services.c
os_specific.c
phello_module.c
prakt
prakt.c
process_syscalls
os_service
os_services
os_services.c

rafl_i_musthofa@RAFLI:~$
prakt.c
process_syscalls
process_syscalls.c
simple_shell.c
simple_shell.c
syscall_test.txt
simple_shell.c
simple_shell.c
syscall_test.txt
tugas.c
shell> ls | grep .c | wc -l
19
shell> echo Halo Dunia > hasil.txt
Halo Dunia
shell> cat hasil.txt
cat: hasil.txt: No such file or directory
shell> cat < hasil.txt
halo saya rafl_i musthofa
halo saya rafl_i musthofa
exit
^C
rafl_i_musthofa@RAFLI:~$
```

Tugas 4

1. Implementasi



2. Dokumentasi

1. Komentar dalam Kode

Tambahkan komentar berikut dalam kode kamu (system_info_ganjil.c) untuk menjelaskan proses parsing dan struktur /proc:

```
// =====  
// PARSING FILE /proc  
// =====  
//
```

// /proc adalah virtual filesystem (VFS) yang berisi informasi real-time tentang kernel dan sistem.

// Setiap file di /proc tidak disimpan di disk, tapi dihasilkan langsung oleh kernel saat dibaca.

```
//
```

// Contoh:

// - /proc/cpuinfo -> Informasi CPU

// - /proc/meminfo -> Informasi RAM

// - /proc/[PID]/ -> Informasi tiap proses (ada file seperti 'status', 'comm')

// - /proc/net/dev -> Statistik jaringan

```
//
```

```
// Program ini membaca file tersebut menggunakan open() + read() (system call level rendah),
// lalu mem-parse teks hasilnya dengan strtok() atau sscanf() untuk mendapatkan nilai spesifik.
//
// Contoh parsing:
// if (sscanf(line, "MemTotal: %ld kB", &mem_total));
// digunakan untuk mengekstrak angka dari baris yang cocok dengan pola tertentu.
```

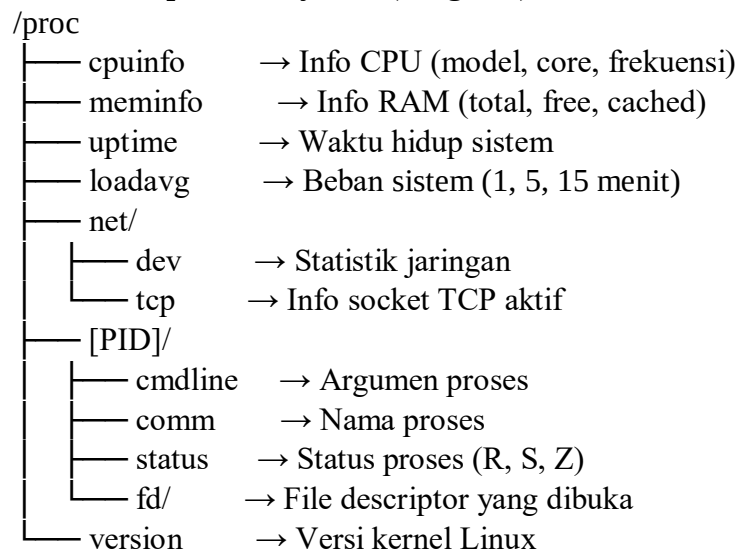
2. Penjelasan Struktur /proc Filesystem

/proc adalah **virtual filesystem** yang menyediakan antarmuka antara **user space** dan **kernel space**.

Karakteristik utama:

- Tidak ada data yang disimpan permanen — semuanya dihasilkan *on the fly* oleh kernel.
 - Setiap file di /proc berisi data tekstual yang bisa dibaca langsung (cat, less).
 - /proc/[PID]/ adalah direktori untuk setiap proses aktif.
-

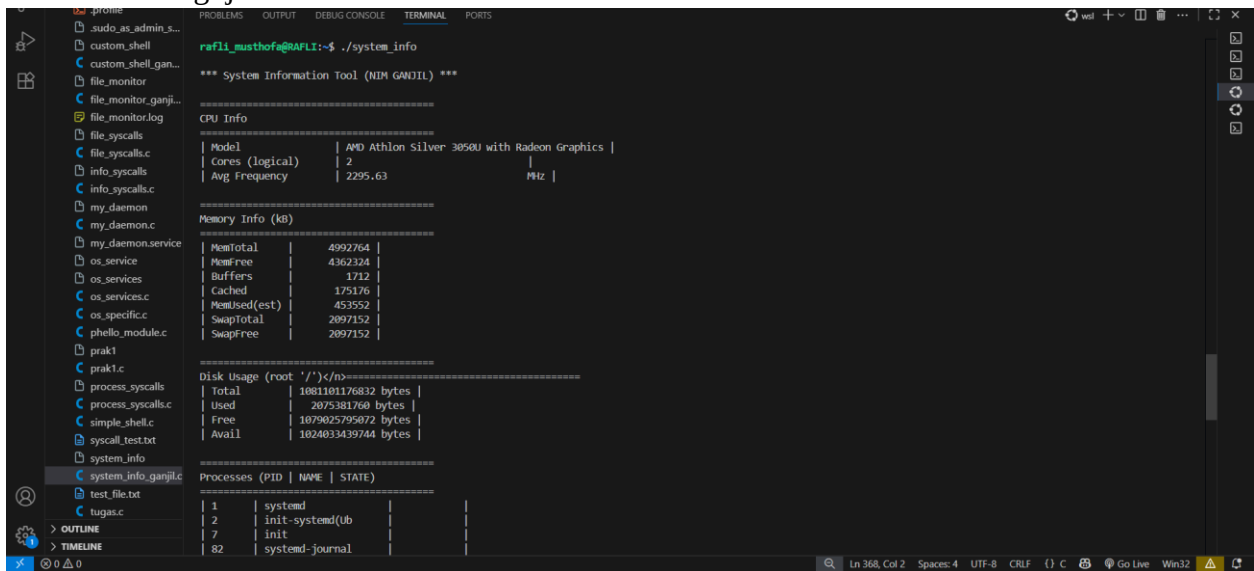
3. Hirarki /proc Filesystem (Diagram)



4. Penjelasan: Virtual Filesystem di Linux

Virtual Filesystem (VFS) adalah lapisan abstraksi yang memungkinkan berbagai sistem file (ext4, procfs, tmpfs, dsb.) bekerja dengan antarmuka yang sama.

3. Pengujian



```
rafli_musthofa@RAFLI:~$ ./system_info

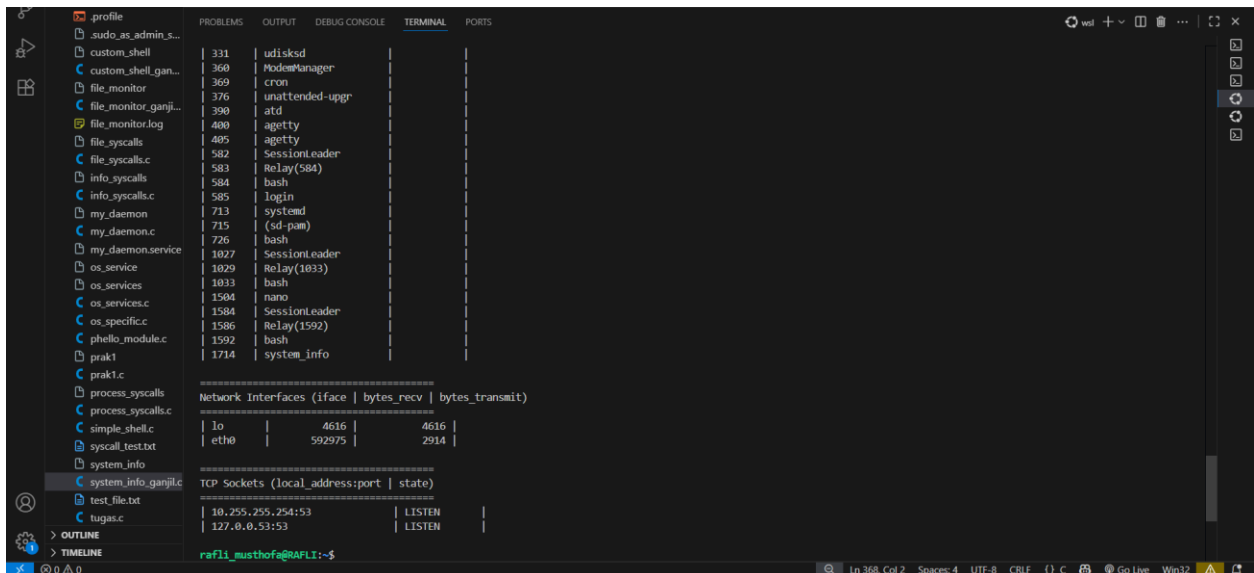
*** System Information Tool (NIH GANJIL) ***

=====
CPU Info
=====
| Model          | AMD Athlon Silver 3050U with Radeon Graphics |
| Cores (logical) | 2                                             |
| Avg Frequency  | 2295.63                                     MHz |
=====

Memory Info (kB)
=====
| MemTotal | 4992764 |
| MemFree  | 4362324 |
| Buffers  | 1712    |
| Cached   | 175176  |
| MemUsed(est) | 463552 |
| SwapTotal | 2097152 |
| SwapFree  | 2097152 |
=====

Disk Usage (root '/')<n>=====
| Total | 108110176832 bytes |
| Used  | 2875381760 bytes  |
| Free  | 1079025795072 bytes |
| Avail | 1024033439744 bytes |
=====

Processes (PID | NAME | STATE)
=====
| 1 | systemd |
| 2 | init-systemd(ub |
| 7 | init |
| 82 | systemd-journal |
=====
```



```
331 | udiskd |
360 | ModemManager |
369 | cron |
376 | unattended-upgr |
390 | atd |
400 |agetty |
405 |agetty |
582 | SessionLeader |
583 | Relay(584) |
584 | bash |
585 | login |
713 | systemd |
715 | (sd-pam) |
726 | bash |
1027 | SessionLeader |
1029 | Relay(1033) |
1033 | bash |
1584 | nano |
1584 | SessionLeader |
1586 | Relay(1592) |
1592 | bash |
1714 | system_info |

Network Interfaces (iface | bytes_recv | bytes_transmit)
=====
| lo | 4616 | 4616 |
| eth0 | 592975 | 2914 |

TCP Sockets (local_address:port | state)
=====
| 10.255.255.254:53 | LISTEN |
| 127.0.0.53:53 | LISTEN |

rafli_musthofa@RAFLI:~$
```

Kesimpulan

- /proc adalah jendela langsung ke **kernel data structures**.
- File dalam /proc bisa dibaca dengan **open() + read()**, tanpa library tambahan.
- Dengan menggabungkan parsing manual, statvfs(), dan loop direktori, program ini dapat menampilkan informasi lengkap sistem.
- Handling error (perror(), continue, validasi buffer) penting untuk mencegah crash pada sistem yang sibuk.